UNIVERSIDADE TÉCNICA DIOGO EUGÊNIO GUILANDE

Faculdade de ciências Tecnológicas

Licenciatura em Informática

3º ano | 2º semestre

Complementos de Programação Visual

Teste 2

RELATÓRIO TÉCNICO

Sistema de gestão de estoque (aplicação desktop)

Discentes:

- Antônio Mucavele
- Leximíbel Langa

Docente:

- Eduardo Nhangala, PhD.

Maputo, aos 28 de Outubro de 2026

Índice

1. Introdução 3
2. Objectivos 4
   2.1. Geral 4
   2.2. Específicos 4
3. Requisitos 5
   3.1. Funcionais 5
   3.2. Regras de negócio 5
   3.3. Não funcionais 5
4. Tecnologias utilizadas 5
5. Arquitectura 6
6. Modelo de dados 6
   6.1. Relação entre tabelas 6
   6.2. Tabela `produtos` 6
   6.3. Tabela `movimentos` 7
7. Aplicação dos conceitos pedidos 7
   7.1. Programação Orientada a Objectos (POO) 7
   7.2. Programação Orientada a Eventos (POE) 8
   7.3. Tratamento de excepções 8
   7.4. Integridade dos dados 8
8. Descrição das funcionalidades 8
   8.1. Dashboard (tela inicial) 8
   8.2. Produtos 9
   8.3. Movimentos 9
   8.4. Alertas 10
9. Testes realizados 10
10. Dificuldades e decisões de desenho 11
11. Limitações e melhorias futuras 11
12. Conclusão 12
13. Referências 13

1. Introdução

Este relatório descreve o desenvolvimento de uma aplicação desktop de gestão de stock que permite registar, consultar, pesquisar, actualizar e eliminar produtos, controlar entradas e saídas, validar estados, identificar produtos abaixo do stock mínimo e guardar os dados numa base de dados SQL.

O enunciado propunha C# com Windows Forms. Com autorização do docente, a solução foi desenvolvida em **Java (Swing)**, plataforma equivalente para interfaces desktop orientadas a eventos.

2. Objectivos

2.1. Geral

Desenvolver uma aplicação desktop que automatize o controlo de stock.

2.2. Específicos

- Implementar o CRUD de produtos com validação de dados.
- Registar entradas e saídas de stock com histórico.
- Gerir automaticamente o estado dos produtos.
- Alertar sobre produtos abaixo do stock mínimo.
- Persistir os dados em MySQL.
- Aplicar POO, programação orientada a eventos (POE) e tratamento de excepções.

3. Requisitos

3.1. Funcionais

Código	Requisito
RF01	Registar, consultar, actualizar e eliminar produtos
RF02	Pesquisar produtos por código, nome ou categoria
RF03	Registar entradas e saídas de stock
RF04	Manter o histórico de movimentos
RF05	Validar e gerir o estado dos produtos (ACTIVO, INACTIVO, ESGOTADO
RF06	Listar produtos com stock abaixo do mínimo
RF07	Apresentar um dashboard com indicadores de stock

3.2. Regras de negócio

- O código do produto é único.
- O stock nunca pode ser negativo.
- A quantidade só varia através de movimentos (rastreabilidade).
- Produto com quantidade 0 passa a ESGOTADO; ao receber stock volta a ACTIVO.
- Produtos INACTIVOS não podem ser movimentados.
- Um produto está em alerta quando `quantidade < stock mínimo`.

3.3. Não funcionais

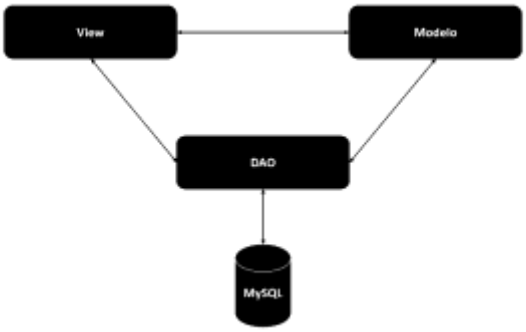
- Usabilidade (avisos claros, cores de alerta),
- Integridade dos dados (transacções),
- Manutenibilidade (arquitectura em camadas)
- Fiabilidade (a aplicação não fecha perante erros).

4. Tecnologias utilizadas

- **Java (JDK [v.25]):** Linguagem de programação
- **Swing:** Interface gráfica
- **NetBeans [v.27]:** Ambiente de desenvolvimento
- **MySQL (XAMPP):** Base de dados
- **JDBC (mysql-connector-j):** Ligação à base de dados
- **GitHub Desktop:** Controlo de versões

5. Arquitectura

A aplicação segue uma arquitectura em camadas:

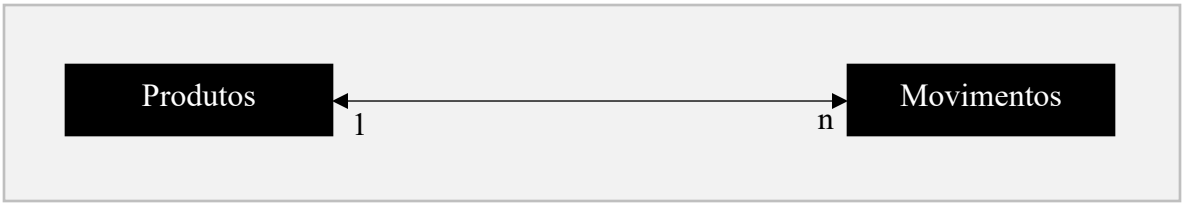


Pacote	Responsabilidade
view	Janelas e painéis (sem SQL)
dao	Todo o acesso à base de dados
modelo	Entidades e regras de negócio
excecao	Excepções personalizadas
util	Classe de ligação `Conexao`

Vantagem: a interface não conhece o SQL e o modelo não conhece a interface, o que facilita alterações e testes.

6. Modelo de dados

6.1. Relação entre tabelas



Um produto tem muitos movimentos. Ao eliminar um produto, os seus movimentos são eliminados (`ON DELETE CASCADE`).

6.2. Tabela `produtos`

Campo	Tipo	Restrições
id	INT	PK, AUTO_INCREMENT
codigo	VARCHAR(20)	NOT NULL, UNIQUE

nome	VARCHAR(100)	NOT NULL
categoria	VARCHAR(50)	
preco	DECIMAL(10,2)	NOT NULL, >= 0
quantidade	INT	NOT NULL, >= 0
stock_minimo	INT	NOT NULL, padrão 5
estado	ENUM	ACTIVO, INACTIVO, ESGOTADO
data_registo	TIMESTAMP	Padrão: data actual

6.3. Tabela `movimentos`

Campo	Tipo	Restrições
id	INT	PK, AUTO_INCREMENT
produto_id	INT	FK – produtos(id)
tipo	ENUM	ENTRADA, SAIDA
quantidade	INT	>= 0
observacao	VARCHAR(200)	
preco	DECIMAL(10,2)	Padrão: data actual

7. Aplicação dos conceitos pedidos

7.1. Programação Orientada a Objectos (POO)

- **Encapsulamento:** os atributos de `Produto` e `Movimento` são privados, acedidos por getters e setters.
- **Validação na origem:** os setters recusam dados inválidos (código vazio, preço negativo, etc.).
- **Enumerações:** `EstadoProduto` e `TipoMovimento` evitam valores inválidos.
- **Regras no modelo:** `adicionarStock()`, `removerStock()` e `isAbaixoDoMinimo()` ficam na classe `Produto`.
- **Herança:** os painéis estendem `JPanel` e a janela estende `JFrame`; as excepções estendem `Exception`.
- **Polimorfismo e interfaces:** a interface `Actualizavel` é implementada por todos os painéis; a janela chama `recarregar()` sem saber o tipo concreto.
- **Abstracção e coesão:** cada classe tem uma responsabilidade (padrão DAO).

7.2. Programação Orientada a Eventos (POE)

O fluxo da aplicação é determinado pelas acções do utilizador:

Evento	Listener	Resultado
Clique em botões	ActionListener	Guardar, eliminar, pesquisar, registar movimento
Enter na pesquisa	ActionListener	Executa a pesquisa
Seleccionar linha da tabela	ListSelectionListener	Preenche o formulário
Mudar de aba	ChangeListener	Recarrega os dados do painel
Alterar produto no combo	ActionListener	Actualiza o stock apresentado

A interface arranca na **Event Dispatch Thread** com `SwingUtilities.invokeLater`.

7.3. Tratamento de excepções

Exceção	Origem	Tratamento
ValidacaoException	Dados inválidos, código duplicado	Aviso ao utilizador
StockInsuficienteException	Saída maior que o stock	Aviso; nada é alterado
SQLException	Falha na base de dados	Mensagem de erro; a aplicação continua
NumberFormatException	Texto em campos numéricos	Convertida em `ValidacaoException’

Técnicas usadas: `try-with-resources` (fecha ligações automaticamente), excepções personalizadas e `rollback` em caso de falha.

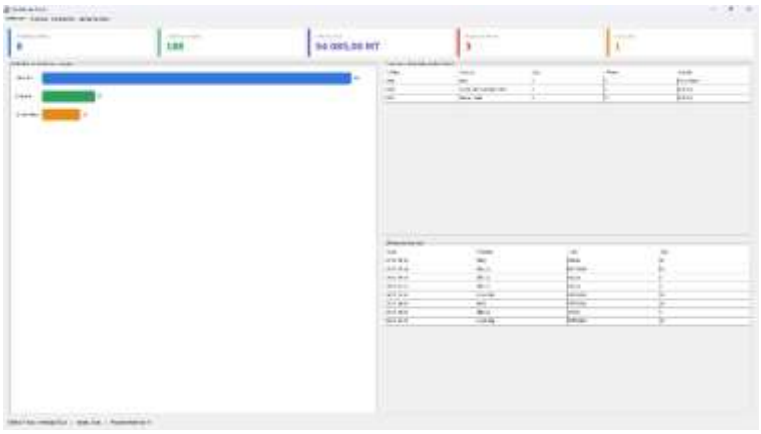
7.4. Integridade dos dados

O registo de um movimento é uma **transacção**: bloqueia a linha do produto (`SELECT ... FOR UPDATE`), actualiza o stock e grava o histórico. Se algum passo falhar, é feito `rollback`. Todas as consultas usam `PreparedStatement`, o que previne SQL Injection.

8. Descrição das funcionalidades

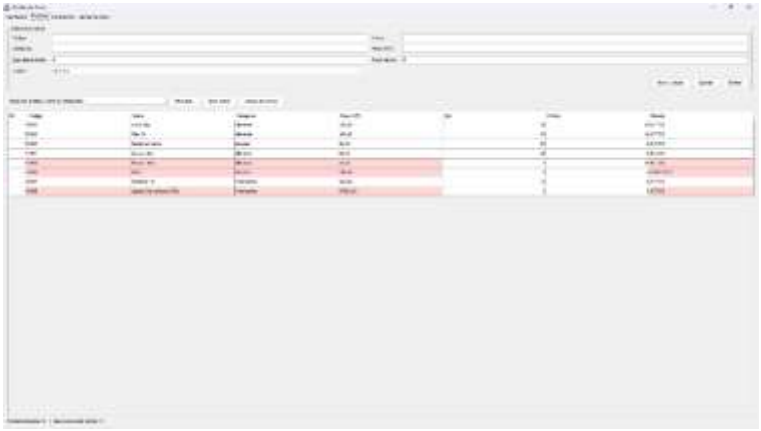
8.1. Dashboard (tela inicial)

Cartões com total de produtos, unidades em stock, valor do stock, produtos abaixo do mínimo e esgotados; gráfico de barras por categoria; produtos críticos; últimos movimentos; resumo de entradas e saídas dos últimos 7 dias.



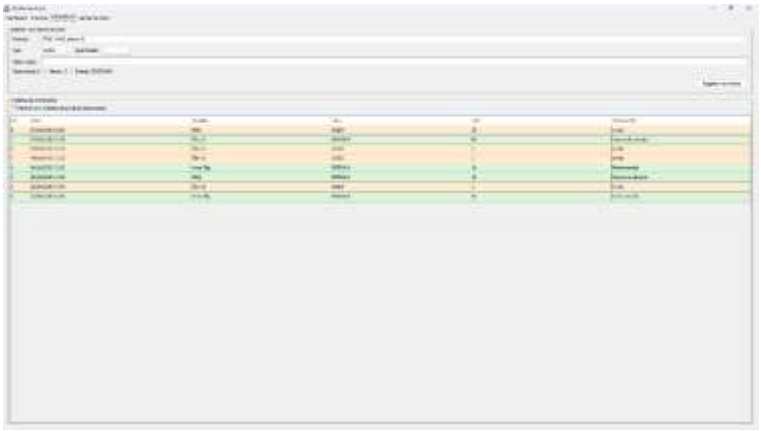
8.2.Produtos

CRUD completo, pesquisa, filtro "abaixo do mínimo" e destaque a vermelho dos produtos em alerta.



8.3. Movimentos

Registo de entradas e saídas com observação, histórico filtrável, cores por tipo e avisos de esgotado ou stock mínimo.



8.4. Alertas

Lista dos produtos a repor, com a quantidade em falta



9. Testes realizados

Nº	Cenário	Resultado esperado	Resultado obtido
1	Registar produto válido	Produto aparece na tabela	✅
2	Registar código duplicado	Aviso de duplicado	✅
3	Nome vazio/preço com letras	Aviso de validação	✅
4	Actualizar preço	Valor alterado	✅
5	Pesquisar "arroz"	Só resultados correspondentes	✅
6	Entrada de 20 unidades	Stock aumenta 20	✅
7	Saída válida	Stock diminui	✅
8	Saída maior que o stock	Aviso; stock inalterado	✅
9	Retirar todo o stock	Estado passa a ESGOTADO	✅
10	Stock abaixo do mínimo	Aviso e entrada em Alertas	✅
11	Produto INACTIVO	Não aparece em Movimentos	✅
12	Eliminar produto	Confirmação e remoção	✅
13	Dashboard após movimentos	Indicadores actualizados	✅
14	MySQL desligado	Mensagem de erro, sem fechar	✅

10. Dificuldades e decisões de desenho

- **Quantidade só por movimentos:** o formulário bloqueia a quantidade ao editar, garantindo rastreabilidade.
- **Estado automático:** ACTIVO/ESGOTADO decidido pelo sistema; INACTIVO é uma decisão manual do utilizador.
- **Transacção nos movimentos:** evita inconsistências entre stock e histórico.
- **Linguagem:** Java em vez de C#, com autorização do docente.

11. Limitações e melhorias futuras

- Autenticação de utilizadores e perfis (administrador, operador).
- Relatórios em PDF/Excel.
- Gestão de fornecedores e vendas.
- Leitura de código de barras.
- Configuração da ligação à BD por ficheiro externo.

12. Conclusão

A aplicação cumpre os requisitos do enunciado: gere produtos e movimentos, valida estados, alerta sobre stock mínimo e persiste os dados em MySQL, aplicando POO, programação orientada a eventos e tratamento de excepções numa arquitectura em camadas.

13. Referências

- Documentação oficial do Java (Oracle): *https://docs.oracle.com/en/java/*
- Documentação do MySQL: *https://dev.mysql.com/doc/*
- Documentação do NetBeans: *https://netbeans.apache.org/*

Anexos

- **Anexo A:** Script SQL completo
- **Anexo B:** Código fonte (ou ligação para o repositório GitHub)
- **Anexo C:** Guia de administração prática